Evaluating Clustering Algorithms and LLM Prompting Strategies for Architecture Recovery in Apache Lucene Codecs

Christopher Seibel Alexander Makejkin Louis Husemann Mikhail Kiselev
Mohammed Danish Shaikh

May 8, 2026

1 Introduction

[Define motivation and objectives. Substantiate the problem context with established literature (e.g., [?]). Formulate the central hypothesis or research goal.]

As software systems evolve, their architectural documentation frequently deviates from the actual implementation. To maintain system integrity and facilitate comprehension, automated architecture recovery techniques are essential. This study evaluates the efficacy of structural clustering algorithms and Large Language Models (LLMs) in extracting and describing the architectural components of an existing software system.
Specifically, this research focuses on the **Codecs** component of the **Apache Lucene** project. By utilizing the **ARCADE** tool suite, we extract system-wide dependencies and isolate component-relevant relationships to generate architectural clusters. The objective is to assess the performance of three established clustering algorithms—**WCA, Limbo, and ACDC**—measuring their structural accuracy through **a2a** (Architecture-to-Architecture) and **Cvg** (Cluster Coverage) metrics.
Furthermore, this study investigates the intersection of structural recovery and generative AI. We evaluate how different prompting techniques influence an LLM's ability to synthesize high-level architectural descriptions from source code. This evaluation utilizes a dual-tier approach: local prototyping with the lightweight `Mistral-7B-Instruct-v0.3` and production-scale execution with the heavyweight `Mistral-Large-3-675B-Instruct-2512`.
Ultimately, this work aims to address the variance in accuracy across different clustering algorithms and determine the impact of prompting strategies on the quality of AI-generated architectural summaries.

2 Study Design

2.1 Research Questions

[Motivation for the individual research questions. Why are these metrics/aspects relevant?]

- **RQ1:** How do different clustering algorithms vary in their ability to accurately determine the architectural components of a system?
- **RQ2:** How do different prompting techniques impact LLM's ability to accurately describe architectural components based on source code?

2.2 Methodology

[Architecture of the experimental setup. Specify the data collection process (e.g., scraping, APIs, existing corpora).]

The experimental setup follows a phased workflow designed to recover the architecture of the Apache Lucene Codecs component through a combination of structural clustering and Large Language Model (LLM) analysis.

2.2.1 Dependency Extraction and Preprocessing

The initial phase involves compiling the Java source code to generate compiled *.jar files. System-wide dependencies are extracted using the ARCADE JavaParser tool, which produces a master .rsf file representing all project relationships. To isolate the scope to the *Codecs* component, the dependencies are filtered such that only relationships where the source or target packages reside within the component's namespace are retained.

2.2.2 Architectural Clustering Algorithms

To address RQ1, three distinct structural clustering algorithms are applied to the filtered dependency graph:

- **Weighted Combined Algorithm (WCA):** A hierarchical clustering approach utilizing edge weights to determine cluster merges.
- **LIMBO:** An information-theoretic algorithm designed to minimize the loss of mutual information during clustering.
- **ACDC:** A heuristic-based algorithm that identifies architectural patterns based on structural connectivity.

Technical implementation details and CLI commands are provided in Appendix A.

2.2.3 LLM-Based Architectural Recovery

To address RQ2, the study implements a dual-stage pipeline designed to verify prompt stability before scaling to the full dataset.

Prototyping Tier: Lightweight Evaluation The initial prototyping phase is conducted using the assigned lightweight model, `Mistral-7B-Instruct-v0.3`. This model is an instruction-fine-tuned version of the Mistral-7B-v0.3 base model, optimized for conversational tasks and complex direction-following. Key technical features utilized during this phase include:

- **Environment:** Prototyping is executed within Google Colab to verify prompt logic stability on smaller samples before full deployment.
- **Technical Capabilities:** The model's native support for function calling and the specialized Mistral tokenizer are leveraged to handle structural architectural data efficiently.

Production Tier: Heavyweight Scaling Upon validation of the prompting logic, the recovery process is scaled to the university's High Performance Computing (HPC) cluster.

- **Model Scaling:** The study utilizes the heavyweight `Mistral-Large-3-675B-Instruct-2512` to process the complete Codecs component of the Lucene project.
- **Execution Framework:** Scaling is managed via Slurm job scheduling.
- **Output Generation:** The primary goal of this tier is to generate concise architectural titles and a high-level descriptive summary for each cluster identified in the structural recovery phase.

2.3 Data Analysis Process

[Detailed explanation of the analysis pipeline. Describe applied algorithms, statistical methods, heuristics, or machine learning models. Ensure reproducibility.]

The data analysis pipeline is structured into three primary phases: structural normalization, quantitative statistical profiling, and comparative similarity evaluation.

2.3.1 Structural Normalization and Metadata Extraction

Raw architectural outputs from the ARCADE toolset are generated in the *.rsf* format, which represents dependencies line-by-line as `depends Source_Package Target_Package`. To facilitate objective comparison, a post-processing utility is utilized to transform these raw files into structured plain-text artifacts. This utility performs the following operations:

- **Sorting:** Clusters are ordered by cardinality (size), and internal elements are sorted alphabetically to ensure deterministic comparison.
- **Metadata Generation:** For each algorithmic run, the utility extracts a statistical profile used to determine the optimality of the clustering parameters.

2.3.2 Quantitative Profiling Metrics

Algorithm selection and parameter tuning are based on a set of distribution metrics that characterize the resulting architecture. These metrics include:

- **Cluster Cardinality:** Total number of identified clusters and the count of singletons (size $= 1$) versus multi-element clusters (size > 1).
- **Central Tendency:** Average size per cluster across the entire system and specifically for clusters with size > 1.
- **Distribution Variance:** Median size for multi-element clusters and the absolute size of the largest identified cluster.

2.3.3 Comparative Similarity Evaluation (RQ1)

To quantify the variance between clustering algorithms, the processed *.rsf* files are analyzed using the **Architecture-to-Architecture** and **Cluster Coverage** metrics.

- **a2a Similarity:** Measures the global structural distance between two architectures, providing a percentage-based similarity score.
- **Cluster Coverage (cvg):** An asymmetric metric measuring the extent to which clusters in one architecture overlap with clusters in another.

A high percentage in these metrics indicates a high degree of structural convergence between different algorithmic approaches.

2.3.4 Qualitative LLM Assessment (RQ2)

For the evaluation of LLM-generated architectural descriptions, a qualitative assessment is conducted. This involves mapping the generated titles and high-level summaries back to the source code elements within each cluster to evaluate semantic accuracy and the impact of different prompting techniques.
TODO in further weeks.

3 Results

[Present empirical findings neutrally, ideally structured by RQ1, RQ2, etc. Aggregate raw data and support with graphs/tables. No interpretation in this section.]

This section presents the quantitative findings derived from the structural clustering of the Apache Lucene Codecs component.

3.1 Algorithmic Profiling and Cluster Distribution (RQ1)

The statistical profiles of the generated architectures for WCA and LIMBO are summarized in Table 1. All algorithms were configured to target a cluster count of 70 to ensure comparability.

Table 1: Quantitative Profiling of Clustering Algorithms (Target: 70 Clusters)

Metric	WCA (UEMNM)	WCA (UEM)	LIMBO
Cluster Cardinality			
Total number of clusters	70	70	70
Number of clusters (size = 1)	48	45	26
Number of clusters (size > 1)	22	25	44
Central Tendency			
Avg. size per cluster	2.70	2.70	2.70
Avg. size (clusters > 1)	6.41	5.76	3.70
Distribution Variance			
Median size (clusters > 1)	6	6	2
Largest cluster size	11	10	41

3.2 Structural Similarity Evaluation

Table 2 illustrates the structural similarity and overlap across the algorithm pairs. For these metrics, WCA with the UEM method was utilized as the baseline.

Table 2: Architecture-to-Architecture (a2a) and Cluster Overlap (cvg) Results

Algorithms (A vs B)	a2a Similarity	cvg (A → B)	cvg (B → A)
LIMBO vs WCA	83.30%	2.0%	2.0%
WCA vs ACDC	80.00%	2.0%	20.0%
LIMBO vs ACDC	77.19%	0.0%	0.0%

4 Discussion

[Interpretation of the results from Section 3. Comparison with the initial hypothesis.]

This section interprets the quantitative findings from the structural recovery phase and evaluates the efficacy of the applied clustering algorithms in accurately determining the architectural components of the Apache Lucene Codecs module.

4.1 Interpretation of Clustering Profiles (RQ1)

An analysis of the clustering profiles indicates that the WCA algorithm, particularly when utilizing the UEMNM measure, yields a more balanced and structurally cohesive architecture

for the given target of 70 clusters. WCA (UEMNM) produces a higher median cluster size (6) and a reasonable maximum cluster size (11), suggesting it successfully identifies moderately sized, functional subsystems within the code.
Conversely, as evidenced by the distribution metrics in Table 1, LIMBO demonstrates an aggressive 'bottom-up' tendency, resulting in a significantly skewed architecture. While targeting the same total cluster count, LIMBO aggregates a massive number of elements into a single monolithic cluster (size 41) while simultaneously generating a high volume of extremely small clusters (median size 2). Qualitative inspection of the elements within this monolith reveals that LIMBO frequently groups structurally unrelated components into this single large cluster, rendering its output highly suboptimal for practical architecture recovery in this specific context.

4.2 Analysis of Structural Similarity and Overlap

As detailed in Table 2, the Architecture-to-Architecture evaluation reveals the highest structural similarity (83.30%) between LIMBO and WCA. In contrast, ACDC has a lower overall similarity score.
However, the global similarity masks severe local discrepancies, which are exposed by the Cluster Coverage metric. The inherent asymmetry of cvg is evident in the comparison between WCA and ACDC (20.0% vs. 2.0% in Table 2), a variance heavily influenced by the differing total cluster counts. Most notably, the coverage between LIMBO and ACDC registers at exactly 0.0%. This complete lack of structural overlap is a direct byproduct of LIMBO's highly skewed distribution; the 41-element monolith (refer back to Table 1) fundamentally misaligns with ACDC's heuristic patterns, failing to meet the minimum percentage thresholds required by the ARCADE cvg tool to register a match.

4.3 Implications for Researchers

[What theoretical insights were gained? How does this advance the state of the art?]

4.4 Implications for Practitioners

[Concrete recommendations for action for software engineers, architects, or management based on the quantitative metrics.]

5 Threats to Validity

[Critical reflection on the study design.]

- **Internal Validity:** An identified bug in the ARCADE Clusterer prevents the correct functionality of the `stopthreshold` argument. Consequently, the algorithms default to generating at least 50 clusters, limiting comparability for smaller target cluster counts.
- **Construct Validity:** *[Do the chosen metrics actually measure what they are intended to?]*
- **External Validity:** *[Are the results generalizable to other datasets?]*

6 Conclusion and Future Work

[Compact synthesis of the work. Identification of technical or conceptual limitations. Outline concrete next steps for future work.]

7 Hours spent per student

Name	W1	W2	W3	W4	W5
Christopher Seibel	1	4	0	0	0
Alexander Makejkin	4	2	0	0	0
Louis Husemann	3	3	0	0	0
Mikhail Kiselev	3	3	0	0	0
Mohammed Danish Shaikh	3	3	0	0	0

A Commands and Technical Reference

A.1 Cluster Generation

WCA:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsf projname=luccodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=WCA measure=<UEMNM or UEM> \
serial=STEPCOUNT serialthreshold=1
```

LIMBO:

```
java -Xmx4096m -jar arcade_core_clusterer.jar language=java \
deps=filtered.rsf projname=luccodecs projversion=9.0.0 \
projpath=output-cluster packageprefix="org.apache.lucene.codecs" \
algo=LIMBO measure=IL \
serial=STEPCOUNT serialthreshold=1
```

ACDC:

```
java -jar arcade_core-ACDC.jar filtered.rsf acdc.rsf
```

A.2 Metrics Evaluation

a2a & cvg :

```
java -jar arcade_core_A2a.jar ./file1.rsf ./file2.rsf
java -jar arcade_core_Cvg.jar ./file1.rsf ./file2.rsf
```